

CS443 Project 1, in Plain English

This project teaches a computer to recognize images in two stages instead of one.

First, it uses a **Hebbian network** to look at many images and learn useful visual patterns without being told the right answer. You can think of this as the system building a set of "feature detectors" that respond to lines, shapes, textures, or color patterns.

Second, it uses a **decoder network** to turn those learned patterns into a final guess, such as which digit appears in an MNIST image or which object category appears in a CIFAR-10 image.

The Main Pieces

- ``image_datasets.py``: loads MNIST or CIFAR-10, normalizes the pixel values, and optionally flattens each image into one long row of numbers.
- ``hebb_net.py``: the bio-inspired part. Neurons compete with each other, one neuron wins, one lower-ranked neuron is inhibited, and the weights update with a Hebbian/Oja-style learning rule.
- ``decoder_nets.py``: contains two small classifiers:
 - ``LinearDecoder``: a simple output layer with softmax for classification.
 - ``NonlinearDecoder``: a tanh-based version that can model more complex patterns.
- ``network.py`` and ``layers.py``: the custom neural-network library used to train and evaluate the decoder networks with TensorFlow.
- The notebooks: walk through each stage of the assignment, from building layers, to training the Hebbian model, to decoding class labels from its activations.

How Data Moves Through the Project

1. Load image data such as MNIST or CIFAR-10.
2. Preprocess the images so the numbers are in a good range for learning.
3. Train the Hebbian network on the images **without labels** so it learns useful features.
4. Save the learned Hebbian weights.
5. Feed images through the Hebbian network to get activations, which are compact feature signals.
6. Train a decoder network on those activations **with labels** so it can predict the correct class.
7. Measure accuracy on validation or test data.

Why This Project Matters

Most beginner image classifiers learn everything at once with backpropagation. This project separates learning into:

- **feature discovery** first, using a biologically inspired rule